Name: Vanshika Wadhwani **D15A/Roll no:15**

EXPERIMENT – 8

AIM:

Design and train a Convolutional Neural Network (CNN) on an image dataset for classification.

THEORY:

1. Dataset Source

- **Dataset:** Fashion-MNIST Dataset
- **Source:** Zalando Research
- **GitHub Link:** https://github.com/zalandoresearch/fashion-mnist

The Fashion-MNIST dataset is a replacement for MNIST and contains images of clothing items used for image classification tasks.

2. Dataset Description

The dataset consists of grayscale images of fashion products categorized into 10 classes.

Dataset Characteristics:

- Total images: 70,000
- Training images: 60,000
- Testing images: 10,000

Image Properties:

- Image size: 28 × 28 pixels
- Color format: Grayscale
- Pixel range: 0 to 255

Classes:

- T-shirt/top

- Trouser
- Pullover
- Dress
- Coat
- Sandal
- Shirt
- Sneaker
- Bag
- Ankle boot

Preprocessing Steps:

- Pixel values normalized to range 0–1
- Images reshaped into 4D tensors:
  (samples, height, width, channels)

3. Mathematical Formulation of CNN

A Convolutional Neural Network processes images using convolution operations.

Convolution Operation:

$Feature\ Map = Input\ Image * Kernel$Feature Map=Input Image∗Kernel

Filters detect features such as edges, textures, and patterns.

Activation Function:

ReLU:

$ReLU(x)$=max(0,x)ReLU(x)=max(0,x)

Pooling Layer:

MaxPooling reduces dimensionality by selecting maximum values from feature maps.

Flatten Layer:

Converts 2D feature maps into 1D vector.

Output Layer:

Softmax function:

$Softmax(xi)=exi∑exjSoftmax(xi)=∑exjexi$

Loss Function:

Sparse Categorical Cross Entropy:

$Loss=−∑yilog(pi)Loss=−∑yilog(pi)$

4. Algorithm Limitations

- Requires high computational power
- Training time increases with deeper networks
- Sensitive to hyperparameters
- May overfit without regularization

5. Methodology / Workflow

- Import required libraries
- Load Fashion-MNIST dataset
- Visualize sample images
- Normalize pixel values
- Reshape data for CNN input
- Design CNN architecture
- Compile model using optimizer and loss function
- Train model with validation data
- Evaluate model using accuracy and confusion matrix
- Visualize predictions and performance

6. Performance Analysis

- CNN achieved high accuracy on test dataset
- Training and validation curves show good learning behavior
- Confusion matrix indicates strong classification performance

Observations:

- Easily distinguishable classes (e.g., shoes, bags) perform well
- Similar classes (e.g., shirt vs t-shirt) show slight confusion

7. Hyperparameter Configuration

- Conv Layer 1: 32 filters, kernel (3×3)
- Conv Layer 2: 64 filters, kernel (3×3)

- Pooling: MaxPooling (2×2)
- Dense Layer: 128 neurons
- Dropout: 0.3
- Output Layer: 10 neurons (Softmax)

Training Parameters:

- Optimizer: Adam
- Loss: Sparse Categorical Crossentropy
- Epochs: 5
- Batch Size: 64

CODE:

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from sklearn.metrics import confusion_matrix, classification_report

# 2. Load Dataset (Fashion-MNIST)
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

# 3. Visualize Sample Images
plt.figure(figsize=(8,6))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title("Label: " + str(y_train[i]))
    plt.axis('off')
plt.show()

# 4. Normalize Data
```

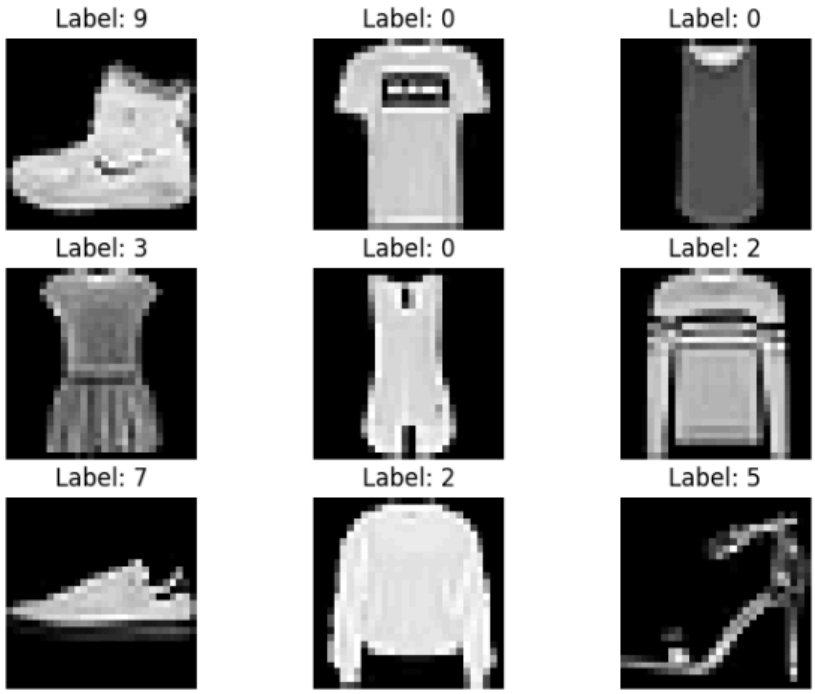
```
X_train = X_train / 255.0
X_test = X_test / 255.0

# 5. Reshape for CNN (IMPORTANT)
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

print("New shape:", X_train.shape)

# 6. Build CNN Model (Same as PDF)
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(10, activation='softmax'))

# Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# 7. Train Model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
```

```
)

# 8. Plot Training Graphs (LIKE PDF)
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Model Loss")
plt.legend()

plt.show()

# 9. Evaluate Model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("\nTest Accuracy:", test_acc)

# 10. Predictions
y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)

# 11. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("CNN Confusion Matrix")
plt.show()

# 12. Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
# 13. Show Predictions (LIKE PDF)
plt.figure(figsize=(10,6))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title("Pred: " + str(y_pred[i]))
    plt.axis('off')
plt.show()
```

OUTPUT

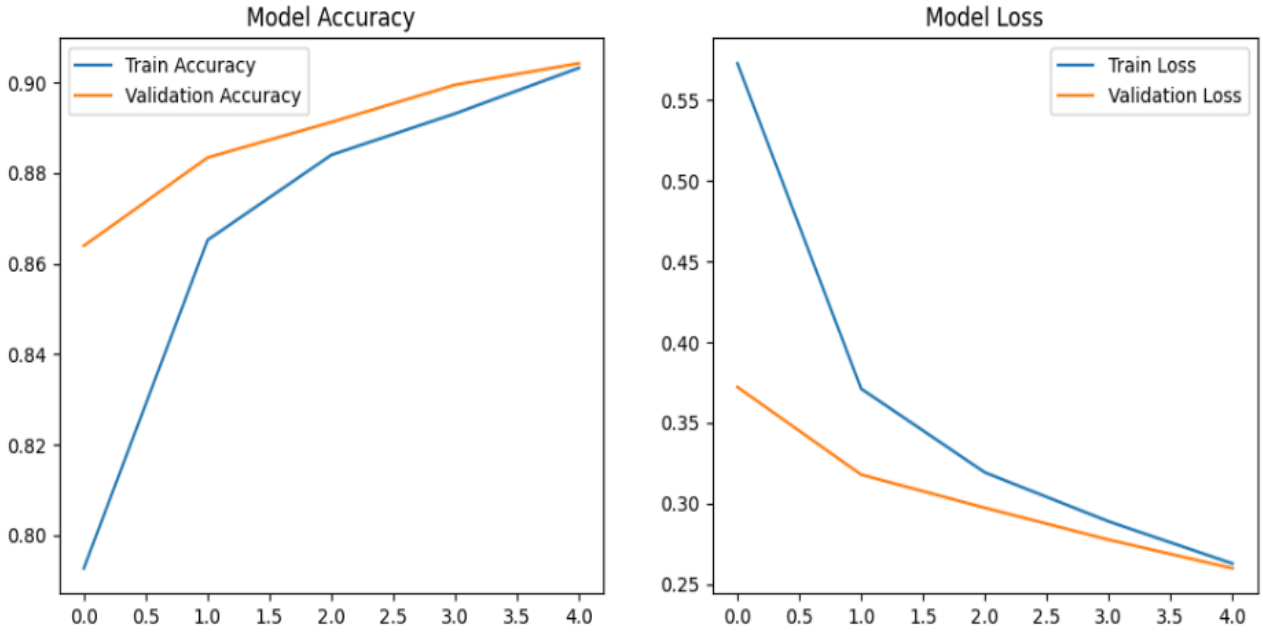
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Training shape: (60000, 28, 28)
Testing shape: (10000, 28, 28)
```



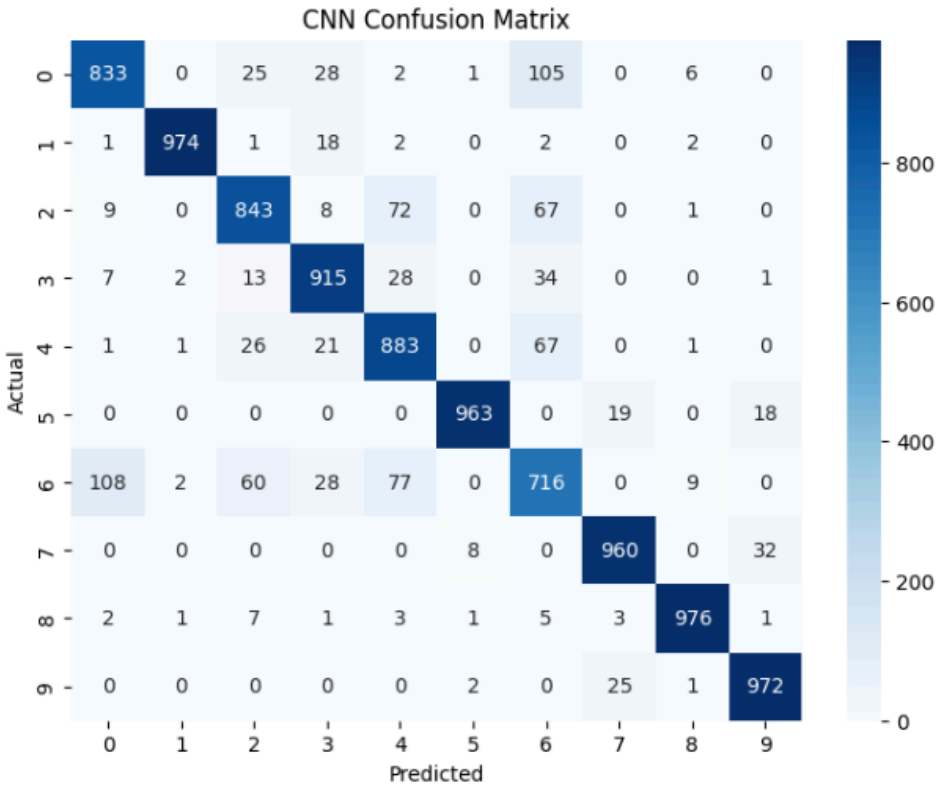
```
New shape: (60000, 28, 28, 1)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_s
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_9"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense_28 (Dense)	(None, 128)	204,928
dropout_2 (Dropout)	(None, 128)	0
dense_29 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 46s 58ms/step - accuracy: 0.7926 - loss: 0.5728 - val_accuracy: 0.8639 - val_loss: 0.3722
Epoch 2/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.8652 - loss: 0.3711 - val_accuracy: 0.8834 - val_loss: 0.3179
Epoch 3/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 82s 55ms/step - accuracy: 0.8840 - loss: 0.3192 - val_accuracy: 0.8913 - val_loss: 0.2973
Epoch 4/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.8931 - loss: 0.2889 - val_accuracy: 0.8995 - val_loss: 0.2776
Epoch 5/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 81s 55ms/step - accuracy: 0.9032 - loss: 0.2628 - val_accuracy: 0.9043 - val_loss: 0.2598
```

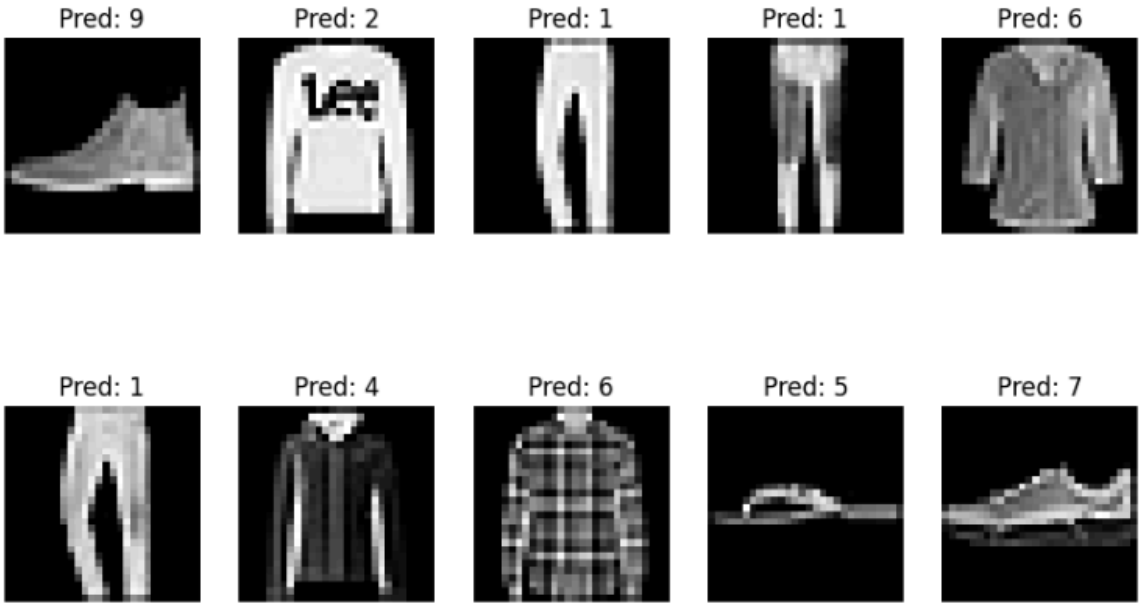



```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9035 - loss: 0.2742

Test Accuracy: 0.9035000205039978
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step
```


CNN Confusion Matrix
Actual
Predicted
0 1 2 3 4 5 6 7 8 9
833 0 25 28 2 1 105 0 6 0
1 974 1 18 2 0 2 0 2 0
9 0 843 8 72 0 67 0 1 0
7 2 13 915 28 0 34 0 0 1
1 1 26 21 883 0 67 0 1 0
0 0 0 0 0 963 0 19 0 18
108 2 60 28 77 0 716 0 9 0
0 0 0 0 0 8 0 960 0 32
2 1 7 1 3 1 5 3 976 1
0 0 0 0 0 2 0 25 1 972
800
600
400
200
0

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.83	0.85	1000
1	0.99	0.97	0.98	1000
2	0.86	0.84	0.85	1000
3	0.90	0.92	0.91	1000
4	0.83	0.88	0.85	1000
5	0.99	0.96	0.98	1000
6	0.72	0.72	0.72	1000
7	0.95	0.96	0.96	1000
8	0.98	0.98	0.98	1000
9	0.95	0.97	0.96	1000
accuracy			0.90	10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000



CONCLUSION

- CNN is highly effective for image classification tasks
- The model successfully learned spatial patterns from fashion images
- Convolution and pooling layers help in automatic feature extraction
- High accuracy demonstrates CNN's capability in computer vision problems
- This experiment confirms the effectiveness of deep learning for real-world image datasets